

Safety-Aware Diffusion Policies via Denoising-Time CBF Filtering for Robotic Manipulation

Abstract—Ensuring collision avoidance while maintaining task progress remains a challenge for diffusion-based manipulation control. Constrained state-space plans remain sensitive to inverse-dynamics and tracking errors during execution, while path-preserving safety filters can stop unsafe motion but cannot redirect a blocked trajectory. To address these limitations, we propose a control barrier function (CBF) safety filter that enforces constraints directly within the action-denoising process of a pretrained diffusion policy. Our method combines trajectory-level projection under discrete-time CBF constraints with task-conditioned denoising to reshape nominal action chunks around obstacles without retraining the policy. At selected denoising steps, intermediate chunks are projected to satisfy CBF constraints and fed back into subsequent denoising steps. The controller executes a short action prefix and replans from updated observations. We evaluate our method on the Can, Square, and Lift tasks from robomimic and on a real UR3 manipulator. Across 300 simulation trials per method, our filter achieves an aggregate joint success-and-safety rate of 42.7%, exceeding the best constrained state-space diffusion baseline by 25.4 percentage points and path-preserving safety filtering by 32.4 percentage points. On Can and Square, denoising-time projection improves joint success and safety by up to 22.5 percentage points over final-output filtering, while adding obstacle-aware CBF constraints yields gains of up to 50.0 percentage points over action bounds alone. UR3 demonstrations further show obstacle-aware passage while the obstacle remains in the workspace. Project page: <https://dasf-project.github.io/>.

I. INTRODUCTION

Visuomotor policies learned from demonstrations are a practical route to complex manipulation. Diffusion policies (DPs) [1] model the conditional distribution of future actions given recent observations through iterative denoising, capturing multimodal behavior from high-dimensional inputs. At deployment, however, the workspace may contain obstacles absent from the demonstrations, as in Fig. 1. Imitation learning alone does not explicitly enforce collision-avoidance constraints in such scenes, while retraining for each obstacle configuration is impractical. This motivates inference-time safety filtering of a frozen policy.

Safety filtering must also account for the temporal structure of DP outputs. Rather than producing a single action, a DP generates an *action chunk*: a sequence of future actions denoised jointly from Gaussian noise. A prefix of H_e actions is executed before replanning from updated observations. Since the chunk encodes temporally correlated task behavior, modifying individual actions can disrupt its coherence. A correction applied only after generation is not reconsidered by the denoising process before execution.

Existing safety filters [2], [3] intervene on policy outputs using control barrier function (CBF) constraints [4], [5],

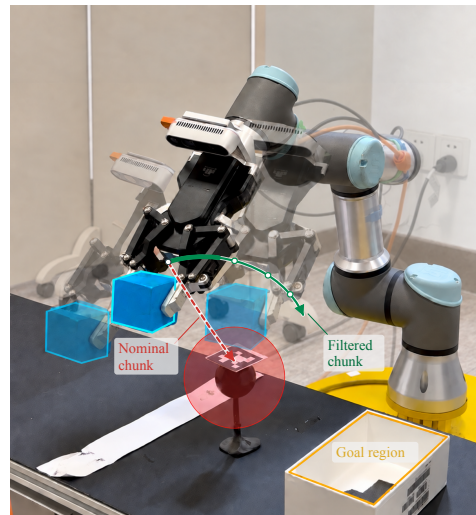


Fig. 1: Motivating example scenario. A robotic manipulator transports a blue block toward the goal region (gold) while navigating around a spherical obstacle. Our method applies CBF projections during diffusion-policy denoising to reshape nominal chunks (red) into obstacle-aware filtered chunks (green).

reachability analysis with a failsafe fallback [6], [7], or per-step filtering of commanded motion [8]. Such approaches can enforce safety under their respective assumptions, but the resulting corrections may depart from the learned task behavior. Path-consistent safety filtering (PACS) [9] addresses this concern by preserving nominal path geometry and modulating progress along it, using set-based reachability analysis to verify safety. However, its path-preserving intervention cannot itself generate a spatial detour: when an obstacle persistently blocks the nominal path, braking can prevent a collision without enabling continued task progress.

Constrained and safe diffusion planners [10], [11] incorporate constraints within denoising, while DDAT [12] projects predicted trajectories to enforce dynamic admissibility. These advances establish constraint-aware diffusion as a promising planning approach. When generated state trajectories are converted into controls through inverse dynamics or tracking, however, their constraint satisfaction also depends on execution accuracy. Our focus is instead deployment-time safety filtering of a pretrained observation-conditioned manipulation policy, enforcing obstacle-avoidance constraints directly on its executable action chunks.

We propose a *denoising-time safety filter* that couples discrete-time CBF projection with task-conditioned action generation. For a DP whose nominal route is obstructed, the filter modifies intermediate action chunks and returns each corrected sample to the remaining reverse-diffusion steps.

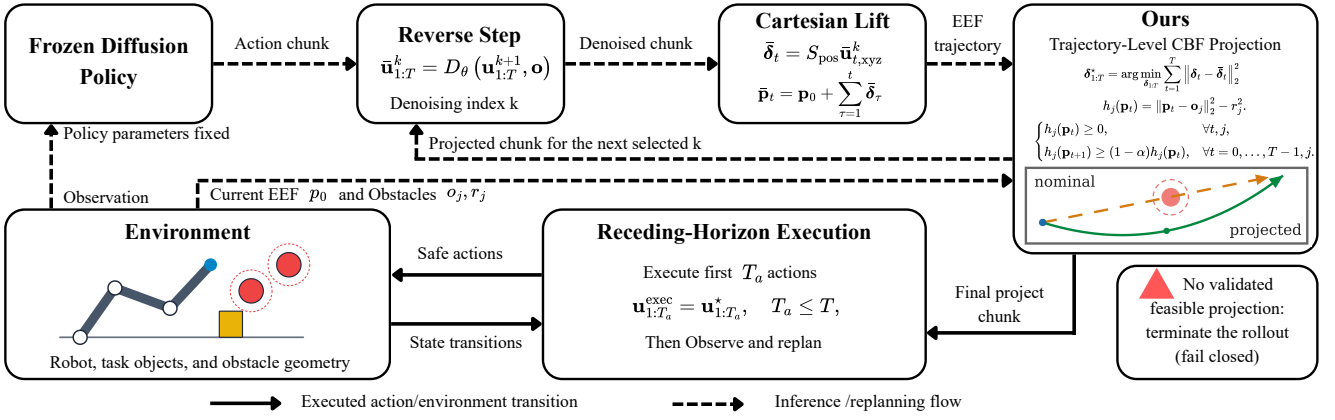


Fig. 2: Overview of the proposed denoising-time safety-filtering pipeline. A frozen diffusion policy generates an action chunk; its future Cartesian motion is rolled out from the measured state and jointly corrected under trajectory-level discrete-time CBF constraints. The corrected chunk is returned to subsequent denoising steps, after which a short action prefix is executed before observation and replanning.

Subsequent denoising can therefore refine the intervention using the learned task prior, rather than executing a correction that the generative process has never reconsidered. This mechanism permits changes to the nominal path geometry without retraining the policy; it does not assume that denoising guarantees recovery of the original training distribution. The overall pipeline of the method is illustrated in Fig. 2.

Concretely, Cartesian end-effector increments are rolled out from the measured end-effector position to obtain the predicted motion. Discrete-time CBF constraints on this rollout define trajectory-level restrictions on the action chunk. During the last K reverse steps, constrained projection adjusts the intermediate chunk subject to these restrictions and action bounds, and the projected chunk is fed into the next denoising step. The controller then executes a short prefix and replans, re-anchoring the constraints to the latest measured state.

Our contributions are threefold: (i) an action-chunk safety formulation that maps discrete-time CBF constraints through a measurement-anchored EEF rollout; (ii) a denoising-time filtering mechanism that feeds corrected chunks back into subsequent reverse steps, enabling obstacle-aware generation without retraining; and (iii) evaluation against DP, Post-CBF, PACS, Constrained Diffusers, and SafeDiffuser on three robomimic tasks, with projection-depth and Bounds-only ablations and UR3 demonstrations. Across 300 simulation trials per method, our filter achieves a 42.7% aggregate joint success-and-safety rate, 9.0 percentage points above Post-CBF, and improves the joint rate over DP, Post-CBF, and PACS on all three tasks.

II. RELATED WORK

A. Diffusion Policies and Action-Chunk Manipulation

Diffusion models [13] entered robotics as trajectory planners that denoise state or state-action sequences and execute them through tracking or inverse dynamics [14], [15]. Diffusion Policy [1] instead denoises a chunk of future *actions* conditioned on recent observations, in the spirit of action chunking [16], and executes a prefix in receding horizon. The object to be constrained is then a chunk of

commanded increments, and the protected quantity is defined only through the rollout of those increments.

B. Safety Filtering of Learned Policies

Safety filters [2], [3] keep a learned controller in a safe set by minimally modifying its commands, using CBFs [4], their discrete-time counterparts [5], [17], or reachability with failsafe maneuvers [6]; they have been applied to manipulator collision avoidance [18] and to per-step end-effector commands of learned visuomotor policies [8]. For chunked policies, RAIL [7] verifies the proposed trajectory by reachability analysis and falls back to a failsafe planner, and PACS [9] brakes along the nominal path when a collision cannot be excluded. All act on a completed chunk, so the generative model plays no part in the correction, and PACS additionally preserves path geometry by design, which excludes rerouting around an obstacle on the path.

C. Safety-Constrained Diffusion Planning

SafeDiffuser [10] treats the denoising update as a dynamical system in diffusion time and enforces CBF conditions on every generated waypoint by a QP at each step, yielding finite-time invariance of the generated state trajectory; it also reports that constraining every step degrades generation quality, and introduces relaxed variants for that reason. Constrained Diffusers [11] formulate constrained Langevin sampling with projected, primal-dual, and augmented-Lagrangian variants; its *projected* variant generates constrained state trajectories that are executed in receding horizon through a learned inverse-dynamics model, making it the closest prior mechanism to ours. DDAT [12] projects generated successor states onto reachable-set approximations for dynamic admissibility, and guidance methods add barrier gradients to the score [19]. Our setting instead treats a pretrained observation-conditioned action policy as the nominal controller and applies inference-time safety filtering directly to its executable action chunks. We enforce trajectory-level CBF constraints on a measurement-anchored rollout of the commanded increments and feed the corrected actions back into the remaining denoising process, avoiding

a separate inverse-dynamics interface between generated states and executed commands. The projection depth K is treated as a design variable rather than constraining the full denoising schedule. Section V compares the resulting safe task completion across three manipulation tasks.

III. PRELIMINARIES

A. Discrete-Time Control Barrier Functions

We consider the discrete-time control system

$$x_{i+1} = F(x_i, u_i), \quad (1)$$

where $x_i \in \mathcal{X} \subseteq \mathbb{R}^n$, $u_i \in \mathcal{U} \subseteq \mathbb{R}^m$, and i indexes control steps with sampling period Δt . Let $\mathcal{X}_s$ be the prescribed safe set, including its boundary, and $\mathcal{X}_u = \mathcal{X} \setminus \mathcal{X}_s$ its unsafe complement. Safety may include a clearance margin, so a state can be unsafe without physical contact.

Definition 1 (Discrete-time control barrier function). *A continuously differentiable function $h : \mathcal{X} \rightarrow \mathbb{R}$ is a discrete-time CBF for (1) if $\mathcal{X}_s = \{x : h(x) \geq 0\}$ and there exists $\gamma \in (0, 1]$ such that every $x \in \mathcal{X}_s$ admits an input $u \in \mathcal{U}$ satisfying*

$$h(F(x, u)) - h(x) \geq -\gamma h(x). \quad (2)$$

This exponential discrete-time CBF condition [5] bounds the permitted one-step decrease of h . Its induced safe set is

$$\mathcal{C} = \{x \in \mathcal{X} : h(x) \geq 0\} = \mathcal{X}_s. \quad (3)$$

If $x_0 \in \mathcal{C}$ and every applied input satisfies (2), then $h(x_{i+1}) \geq (1-\gamma)h(x_i) \geq 0$, establishing forward invariance at the sampled control instants. Applying this result to a robot requires specifying the prediction model, protected geometry, and admissible inputs.

B. Action Diffusion Policies

A pretrained diffusion policy (DP) [1] uses a denoising diffusion probabilistic model (DDPM) [13] to generate action sequences conditioned on recent observations $o(t)$. Let $A^k \in \mathbb{R}^{H \times d_a}$ denote the normalized sequence at reverse step k , with prediction horizon H and action dimension d_a . Starting from $A^N \sim \mathcal{N}(0, I)$, the policy performs N reverse updates, indexed by $k = N, \dots, 1$. For a DDPM schedule α_k and $\bar{\alpha}_k = \prod_{\ell=1}^k \alpha_\ell$, the noise-prediction network ϵ_θ gives

$$\bar{A}^{k-1} = \frac{1}{\sqrt{\alpha_k}} \left(A^k - \frac{1 - \alpha_k}{\sqrt{1 - \bar{\alpha}_k}} \epsilon_\theta(A^k, k, o(t)) \right) + \sigma_k z_k. \quad (4)$$

Here $z_k \sim \mathcal{N}(0, I)$, $\sigma_1 = 0$, and we denote this update by $\bar{A}^{k-1} = \mathcal{D}_\theta(A^k, k, o(t))$. Unfiltered DP sets $A^{k-1} = \bar{A}^{k-1}$ at every step and outputs A^0 .

An affine map $\mathcal{N}_a$, fitted on the training actions, normalizes controller-space commands; $\mathcal{N}_a^{-1}$ decodes them. If the observation horizon is H_o , the next executable action occurs at index $\ell_0 = H_o - 1$. We therefore extract the future slice $\ell_0, \dots, H - 1$, of length $H_f = H - H_o + 1$. Only its first $H_e \leq H_f$ actions are executed before replanning. Throughout, t denotes a control step at which replanning occurs, j indexes future actions within a chunk, and k indexes denoising rather than physical time.

IV. METHOD

The complete pipeline has been shown in Fig. 2. At each replanning step, the frozen diffusion policy generates an action chunk, whose future Cartesian motion is constrained by a trajectory-level discrete-time CBF projection. The corrected chunk is fed back into subsequent denoising steps; after the final projection, the controller executes a short prefix and replans from a new observation.

A. Safety Filtering Formulation

We seek a safety filter for a frozen DP controlling a manipulator around spherical obstacles. Let $p_i \in \mathbb{R}^3$ be the measured EEf position and ξ_i the center of its protected geometry. In the point model, $\xi_i = p_i$; for a tool-fixed sphere, $\xi_i = p_i + R_i b$, where R_i is the tool orientation and b its fixed center offset. An obstacle $o \in \mathcal{O}$ has center c_o and effective protection radius r_o , including obstacle inflation and the protected sphere radius. The barrier is

$$h_o(\xi) = \|\xi - c_o\|_2^2 - r_o^2. \quad (5)$$

The protected center must remain in $\mathcal{C}_{\text{eff}} = \{\xi : h_o(\xi) \geq 0 \forall o \in \mathcal{O}\}$. This specifies EEf protection rather than whole-body collision avoidance.

A controller-space action $u_i = (a_i, \omega_i, g_i)$ consists of translation, axis-angle rotation, and gripper commands. The positive diagonal scale matrix S maps translation commands to physical increments $d_i = S a_i$. The nominal translation model is $p_{i+1} = p_i + d_i$; actual motion can differ through tracking error. Our objective is to modify the generated chunk so that its predicted motion satisfies the DCBF conditions while retaining task behavior. We enforce these conditions over the future chunk and execute a short prefix. The obstacle configuration is held fixed during each planning and execution interval; the geometry is measured again at replanning.

B. Action-Chunk Safety Constraints

Given an intermediate sample A , let $V = \mathcal{N}_a^{-1}(A)$. For $j = 0, \dots, H_f - 1$, the proposed physical increment is $d_j = S V_{\ell_0+j, \text{xyz}}$. Thus, projection acts on the entire future slice after temporal alignment, rather than on entries associated with past observations. The measurement-anchored rollout is

$$\hat{p}_{j|t}(D) = p_t + \sum_{\ell=0}^{j-1} d_\ell, \quad j = 0, \dots, H_f, \quad (6)$$

where $D = (d_0, \dots, d_{H_f-1})$. All positions and displacement bounds are expressed in meters. The scale S accounts for the simulation controller's action scaling; $S = I$ for hardware commands already decoded into meters.

For point protection, $\hat{\xi}_{j|t} = \hat{p}_{j|t}$. For a tool sphere, we use $\hat{\xi}_{j|t} = \hat{p}_{j|t} + \hat{R}_{j|t} b$. Starting from measured R_t , the orientation rollout applies the proposed axis-angle increments after componentwise limiting and rotation-norm limiting. Each bounded incremental rotation is applied by left multiplication in the robot base frame. These orientations are fixed during each translation optimization. The effective radius is $r_o =$

$\mu\rho_o + \rho_g$, where ρ_o is the supplied obstacle-side radius, μ its inflation factor, and ρ_g the protected geometry radius. The simulation point model sets $b = 0$ and $\rho_g = 0$.

Applying the DCBF condition to successive predicted centers gives

$$h_o(\hat{\xi}_{j+1|t}) \geq (1 - \gamma)h_o(\hat{\xi}_{j|t}). \quad (7)$$

The barrier is evaluated along the rollout induced by the candidate increments, anchored at the current measurement. Consequently, these constraints couple future actions across physical control steps. They do not compare consecutive diffusion samples.

The admissible displacement set includes both action-box and step-norm limits:

$$\mathcal{D} = \{d : \underline{d} \leq d \leq \bar{d}, \|d\|_2 \leq d_{\max}\}. \quad (8)$$

Let $\mathcal{W}$ denote the configured Cartesian workspace, with $\mathcal{W} = \mathbb{R}^3$ when no workspace bounds are used. For $m \leq H_f$, define

$$\mathcal{A}_s^m = \{D \in \mathcal{D}^{H_f} : C_j(D) \forall j < m\}, \quad (9)$$

where C_j comprises $\hat{p}_{j+1|t} \in \mathcal{W}$, $h_o(\hat{\xi}_{j+1|t}) \geq 0$, and (7) for every obstacle. Dependence on the measured scene and fixed orientation rollout is implicit. Then $\mathcal{A}_s^{H_f} \subseteq \mathcal{A}_s^{H_c}$: enforcing the full future chunk also constrains its executed prefix.

C. Joint Action-Chunk Projection

We correct all future translation increments together to minimize their total deviation from the proposed chunk. Write $\zeta_t = (p_t, R_t, \mathcal{O}_t)$ for the measured geometric context.

Definition 2 (Joint action-chunk CBF projection). *Given A and ζ_t , decode the proposed increments D as above and solve the constrained correction problem*

$$\begin{aligned} & \underset{D'}{\text{minimize}} && \sum_{j=0}^{H_f-1} \|d'_j - d_j\|_2^2 \\ & \text{subject to} && \underline{d} \leq d'_j \leq \bar{d}, \quad \|d'_j\|_2^2 \leq d_{\max}^2, \\ & && \hat{p}_{j+1|t}(D') \in \mathcal{W}, \\ & && h_o(\hat{\xi}_{j+1|t}(D')) \geq 0, \\ & && h_o(\hat{\xi}_{j+1|t}(D')) \geq (1 - \gamma)h_o(\hat{\xi}_{j|t}(D')). \end{aligned} \quad (10)$$

Constraints hold for all $j = 0, \dots, H_f - 1$ and $o \in \mathcal{O}_t$. Given the returned correction D' , form V' by replacing only $V_{\ell_0+j,xyz}$ with $S^{-1}d'_j$. The complete operator is

$$\Pi_{\text{CBF}}(A; \zeta_t) = \mathcal{N}_a(V'). \quad (11)$$

This decode–correct–encode operation preserves the past-aligned entries and the proposed orientation and gripper components. For sphere geometry, prediction uses the bounded rotation commands that the controller will execute. Geometry and the reference increments are recomputed from each new denoising sample.

Problem (10) uses the exact nonlinear barrier residuals and is generally nonconvex. The implementation jointly optimizes $3H_f$ variables using sequential least squares programming (SLSQP). Here, *projection* denotes a numerically

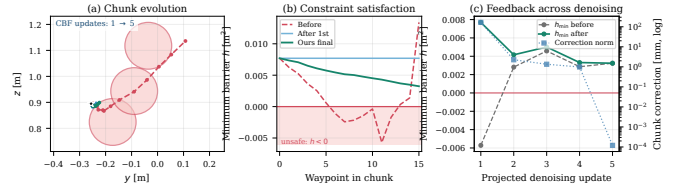


Fig. 3: Denoising-time CBF correction in a representative Can trial. The first projection raises $h_{\min}$ from -5.7×10^{-3} to $7.7 \times 10^{-3} \text{ m}^2$. The correction norm decreases from 1.66×10^{-1} to $2.26 \times 10^{-3} \text{ m}$ at the next projected update and becomes negligible by the final update, which retains $h_{\min} = 3.2 \times 10^{-3} \text{ m}^2$.

feasible correction; global optimality and a unique minimizer are not assumed. Although the explicit clearance inequality follows from the DCBF condition when the initial center is safe, the solver retains and checks it separately.

Within a denoising loop, the preceding corrected chunk provides a warm start. The first call uses a protected-center holding candidate, which is zero translation for the point model. Initialization is limited to the action box and step-norm bound. The solver checks clearance, DCBF, step-norm, and any workspace residuals with tolerance 10^{-6} in their respective units. If optimization fails, the initialization is accepted only if it passes these checks; otherwise the projection reports failure. A holding candidate need not be feasible for offset sphere geometry because compensating rotation can exceed translation or workspace limits.

D. Integration into Denoising

At reverse step k within a replanning loop, the frozen policy computes

$$\bar{A}^{k-1} = \mathcal{D}_\theta(A^k, k, o(t)), \quad (12)$$

using (4). We apply the correction operator during the final K reverse updates:

$$A^{k-1} = \begin{cases} \bar{A}^{k-1}, & k > K, \\ \Pi_{\text{CBF}}(\bar{A}^{k-1}; \zeta_t), & k \leq K. \end{cases} \quad (13)$$

The corrected sample becomes the input to the next reverse step. Subsequent denoising can therefore refine a safety-corrected chunk using the observation-conditioned task prior. This allows spatial changes to the nominal motion, although neither task completion nor recovery of the training distribution is guaranteed. Setting $K = 0$ gives unfiltered DP; $K = 1$ gives Post-CBF, which corrects only the final output.

For $K \geq 1$, the last operation is a projection, so a feasible returned chunk satisfies the rollout constraints before execution. Earlier projections determine how safety correction interacts with generation. Their benefit to safe task completion is evaluated empirically in Section V-B; the nominal safety argument below depends on final feasibility.

Figure 3 illustrates how subsequent denoising refines the projected chunk, with progressively smaller corrections and a positive final barrier margin.

E. Receding-Horizon Execution and Safety

Algorithm 1 summarizes the controller. The observation and measured geometry are fixed during each denoising loop.

After executing the first H_e decoded actions, the controller acquires a new observation and re-anchors the next rollout.

Algorithm 1 Denoising-time CBF filtering for diffusion policies

Require: Frozen policy ϵ_θ ; reverse steps N ; projection depth $K \leq N$; execution horizon H_e ; projection Π_{CBF}

```

1:  $t \leftarrow 0$ ;  $\text{success} \leftarrow \text{False}$ 
2: while not  $\text{success}$  do                                ▷ Policy updates
3:    $o(t) \leftarrow \text{GETOBSERVATION}(t)$ 
4:    $\zeta_t \leftarrow \text{GETSCENEGEOMETRY}(o(t))$ 
5:    $A^N \sim \mathcal{N}(0, I)$ 
6:   for  $k = N, \dots, 1$  do                                ▷ Denoising steps
7:      $A^{k-1} \leftarrow \text{DENOISE}(A^k, k, o(t); \epsilon_\theta)$ 
8:     if  $k \leq K$  then                                    ▷ Final  $K$  updates
9:        $A^{k-1} \leftarrow \Pi_{\text{CBF}}(A^{k-1}; \zeta_t)$ 
10:    end if
11:  end for
12:   $U \leftarrow \text{DECODEFUTURE}(A^0)$ 
13:   $\text{EXECUTEACTIONS}(U_{0:H_e-1})$ 
14:   $t \leftarrow t + H_e$ 
15:   $o(t) \leftarrow \text{GETOBSERVATION}(t)$ 
16:   $\text{success} \leftarrow \text{CHECKSUCCESS}(o(t))$ 
17: end while
```

Proposition 1 (Nominal executed-prefix safety). *Suppose $\xi_t \in \mathcal{C}_{\text{eef}}$, obstacles remain fixed, and the final returned chunk satisfies (10) exactly. If its executed prefix realizes the predicted protected-center motion, then $\xi_{t+j} \in \mathcal{C}_{\text{eef}}$ for every $j = 0, \dots, H_e$.*

Proof. Full-chunk feasibility implies the DCBF inequality at every step of the prefix. Exact realization gives $\xi_{t+j} = \xi_{j|t}$. Applying (7) recursively yields, for every obstacle,

$$h_o(\xi_{t+j}) \geq (1 - \gamma)^j h_o(\xi_t) \geq 0. \quad (14)$$

The claim follows from $0 \leq 1 - \gamma < 1$ and initial safety. $\square$

Repeating this argument at replanning instants gives nominal sampled safety whenever a feasible chunk is available and the assumptions continue to hold. For the static point model, zero increments establish feasibility if the initial position lies in the workspace and $0 \in \mathcal{D}$; this does not imply that the numerical solver always converges or that the task progresses.

Proposition 1 states an exact-model result. Numerical acceptance permits small constraint residuals, and sensing errors, tracking errors, or obstacle motion can invalidate the predicted clearance. The hardware interface additionally checks the executable prefix after command limiting. These checks do not establish robustness to unbounded errors, collision avoidance between samples, or whole-body safety. Task completion and physical safety are therefore also assessed through the experiments.

V. EVALUATION

We evaluate safe task completion through simulation comparisons, ablation studies, and hardware demonstrations.

Experimental Setup. We evaluate on three robomimic benchmarks—Can, Square, and Lift—with obstacles placed along the manipulation corridor. Can requires object transport, Square requires precise placement, and Lift tests grasping and lifting. Within each paired scene, methods share

TABLE I: Implementation details of the simulation and real-robot policies.

Hyperparameter	DP (sim)	DP (real)	PD	RoS
Training demos (Can)	300	100	300	300
Generative model	Diffusion	Diffusion	Diffusion	Diffusion
Architecture	U-Net	U-Net	U-Net	U-Net
Prediction horizon H	16	16	100	128
Execution horizon H_e	4	4	4	4
Denoising steps N	100	100	200	1000
Extra denoising updates	0	0	50	0
Sampling period Δt (ms)	50	100	50	50

H counts predicted actions for DP and states for the other models. H_e counts executed actions, except for RoS, which executes waypoints with at most 5 control steps per waypoint.

the initial task state, object and obstacle placements, and controller scales. Each method is evaluated on 100 scenes per task, with a budget of 1000 control steps at 20 Hz. All obstacles have a 3 cm radius; Can contains three obstacles, while Square and Lift contain two. The learned backbones exhibit broadly comparable nominal task competence, supporting a meaningful comparison of obstacle-handling methods.

Baselines. We compare five baselines:

- **Diffusion Policy (DP)** [1]: generates observation-conditioned action chunks without safety filtering.
- **Post-denoising CBF filtering (Post-CBF)**: projects only the final denoised chunk, without correction feedback. We include it in simulation, ablation, and hardware comparisons.
- **Path-consistent safety filtering (PACS)** [9]: brakes and resumes execution along the nominal path, adjusting progress without changing path geometry.
- **SafeDiffuser (RoS)** [10]: uses the robust-safe variant to constrain state diffusion; Cartesian waypoint tracking and inverse-dynamics gripper control execute the trajectory.
- **Constrained Diffusers (PD)** [11]: uses the primal-dual variant with learned inverse-dynamics execution, tested at $H_e = 4$.

In both simulation and hardware comparisons, DP, PACS, Post-CBF, and Ours share the same frozen DP checkpoint within each task and differ only in their safety mechanism. In simulation, Ours projects the final $K = 5, 25, 25$ denoising steps on Can, Square, and Lift, respectively. For hardware, Ours projects the final denoising steps ($K = 25$) and uses a 6 cm obstacle-side constraint radius. All learned parameters remain frozen during evaluation. Table I summarizes the simulation and hardware policy settings.

A. Simulation Study

Evaluation metrics. We report (1) *Safe*, indicating that no physical contact with any obstacle occurs throughout an episode; (2) *Success*, indicating task completion within the specified time; and (3) *Success and Safe*, requiring both conditions to hold in the same episode. Success uses the native predicate for Can and Square; Lift requires the cube’s lowest contact point to rise more than 1 mm above the table. The EEf constraint radius is set to 9 cm on Can, Square, and Lift.

TABLE II: Simulation rates (%) over 100 trials per task. Bold marks the best value per column; — denotes unavailable results.

Method	Safe			Success			Success & Safe		
	Can	Sq.	Lift	Can	Sq.	Lift	Can	Sq.	Lift
Ours	95	58	15	77	53	61	76	37	15
DP	21	9	5	45	27	76	21	8	5
Post-CBF	86	51	5	76	55	45	63	33	5
PACS	59	89	22	29	12	66	17	9	5
PD	49	2	29	18	3	18	17	0	18
RoS	45	25	15	46	13	65	34	5	13

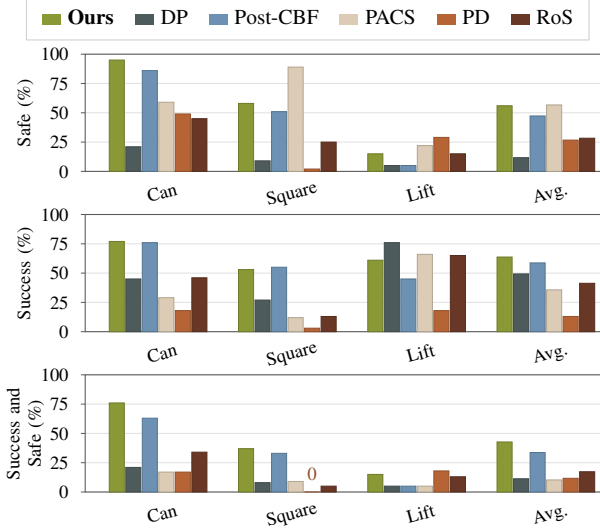


Fig. 4: Comparison of safety, task success, and their joint attainment across three simulated manipulation tasks.

Evaluation results. Figure 4 compares the three metrics by task; Avg. pools the 100-trial batches across Can, Square, and Lift.

Performance of policy filters. Table II and Fig. 4 show that our method achieves the highest aggregate Success and Safe rate: 42.7%, compared with 11.3% for DP, 33.7% for Post-CBF, 10.3% for PACS, 11.7% for PD, and 17.3% for RoS. Relative to DP, aggregate safety increases from 11.7% to 56.0%, while task success increases from 49.3% to 63.7%. Thus, the gain in safe completion does not arise from simply suppressing task execution.

Our method attains the best joint rates on Can and Square, at 76% and 37%, respectively. It exceeds Post-CBF by 13 and 4 percentage points on these tasks, supporting correction during denoising rather than only at its output. On Lift, the joint rate is 15%, compared with 5% for DP, Post-CBF, and PACS, 13% for RoS, and 18% for PD. Although PD performs better on Lift and PACS attains higher safety on Square, our method offers the strongest aggregate safe task completion and improves the joint rate over DP, Post-CBF, and PACS across all three tasks.

B. Ablation Study in Simulation

Following the simulation comparison, we investigate the contribution of denoising-time projection to safe task completion through ablations on Can and Square.

Evaluation protocol. On Can and Square, we vary the final

TABLE III: Success and Safe (%) for 40 paired trials per configuration. K denotes projected denoising steps, and H_e denotes executed actions per replan.

Task	$H_e \backslash K$	0	1	5	10	25	50	100
Can	1	7.5	72.5	82.5	67.5	60.0	62.5	67.5
	2	10.0	75.0	60.0	77.5	72.5	70.0	67.5
	4	22.5	70.0	92.5	67.5	75.0	67.5	57.5
	8	30.0	80.0	67.5	77.5	55.0	57.5	65.0
Square	1	10.0	32.5	30.0	35.0	27.5	25.0	25.0
	2	10.0	35.0	27.5	37.5	40.0	32.5	35.0
	4	12.5	27.5	37.5	35.0	42.5	32.5	30.0
	8	10.0	25.0	20.0	20.0	22.5	32.5	27.5

projected denoising steps, $K \in \{0, 1, 5, 10, 25, 50, 100\}$, and the actions executed before replanning, $H_e \in \{1, 2, 4, 8\}$. Here $K = 0$ is unfiltered DP [1]; $K = 1$ is Post-CBF, which projects the final action chunk once after denoising. The ablation scene set is independent of the main simulation comparison. Each setting uses 40 paired scenes, 100 reverse steps per plan, and a 1000-step rollout budget. The policy, scenes, and 9 cm EEF protection distance are fixed within each task, isolating projection depth and feedback frequency without changing the denoising budget.

Effect of denoising-time projection. Projecting within the denoising process can substantially improve Success and Safe over filtering only the final chunk (Table III). At matched execution horizons, the gains over Post-CBF reach 22.5 percentage points on Can and 15.0 points on Square. These results support our choice to incorporate safety corrections during denoising, allowing subsequent denoising steps to refine the corrected chunk rather than applying a single terminal projection.

Interaction with the execution horizon. The benefit depends on the projection depth and execution horizon: projecting more reverse steps does not necessarily improve safe completion, and Post-CBF remains the best observed setting on Can at $H_e = 8$. Thus, the ablation demonstrates the effectiveness of denoising-time projection under suitable settings, rather than a monotonic benefit from increasing the number of projections.

Contribution of obstacle constraints. We compare Bounds-only with the full filter at $K = 25$ and $H_e = 4$ on the same 40 scenes per task. Bounds-only retains the action-box and per-step translation-norm limits and feeds the bounded chunks into subsequent denoising steps, but removes both obstacle-clearance and discrete-time CBF constraints. The checkpoint, initial states, obstacle placements, sampling seeds, and denoising budget are matched. Safe here uses the same no-contact criterion as the projection-depth ablation.

Table IV shows that Bounds-only yields joint rates close to DP: 25.0% versus 22.5% on Can and 12.5% for both on Square. Adding the obstacle constraints raises the joint rate to 75.0% and 42.5%, respectively, gains of 50.0 and 30.0 percentage points over Bounds-only. Safe rates also increase from 25.0% to 95.0% on Can and from 15.0% to 57.5% on Square. These matched comparisons support the contribution of obstacle-aware safety constraints beyond action limiting

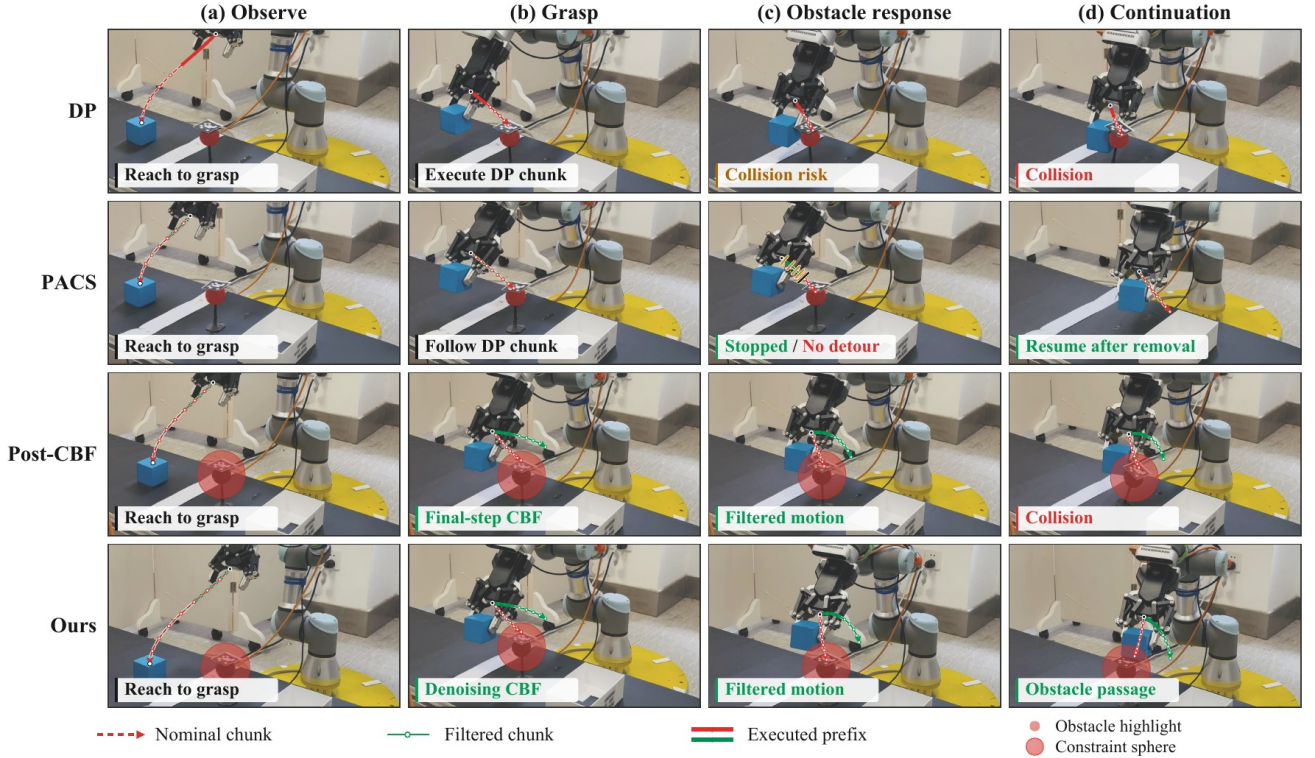


Fig. 5: Real-robot comparison. DP and Post-CBF contact the obstacle. PACS stops along the nominal path and resumes after obstacle removal; Ours illustrates obstacle-aware passage. Red/green curves denote nominal/filtered action chunks, with thick execution prefixes. Amber stripes indicate PACS braking. Red shading highlights physical obstacles (DP/PACS) or constraint spheres (Post-CBF/Ours). Overlays are schematic; the final frames do not establish task completion.

TABLE IV: Effect of obstacle constraints on safety and task completion (% , 40 paired scenes per task).

Task	Method	Safe	Success	Success and Safe
Can	DP	22.5	55.0	22.5
	Bounds-only	25.0	45.0	25.0
	Ours	95.0	75.0	75.0
Square	DP	12.5	37.5	12.5
	Bounds-only	15.0	35.0	12.5
	Ours	57.5	60.0	42.5

during denoising in the tested configurations.

C. Real-Robot Demonstration

Platform and demonstration collection. We deploy the method on a UR3 manipulator with a DH Robotics AG95 gripper, which grasps a blue block and transports it to a target box. Observations come from one Intel RealSense D435i RGB camera mounted on the robot wrist, logged together with the robot state at 10 Hz. All training data are collected by kinesthetic teaching: with the controller in a gravity-compensated, backdrivable mode, a human operator holds the arm near the wrist and guides it through the whole task. Nothing is scripted or teleoperated, so the path, the velocity profile and the grasp pose are whatever the operator produced on that trial. Each demonstration records the RGB images, the joint positions $q_i \in \mathbb{R}^6$ and velocities $\dot{q}_i \in \mathbb{R}^6$, the forward-kinematic end-effector pose (p_i, R_i) , and the gripper state. We collected the 100 demonstrations of Table I, none of which contains an obstacle, so the scenario below lies outside

the demonstrated distribution by construction and tests the filter rather than the policy.

Training data and policy. Trajectories are resampled to $\Delta t = 100$ ms and converted to controller-space actions. Hardware commands are already expressed in meters, so $S = I$ in Section IV-A and the translation command is the demonstrated increment itself, $a_i = d_i = p_{i+1} - p_i$, with ω_i the axis-angle vector of $R_{i+1}R_i^\top$, g_i the logged gripper state, and $\mathcal{N}_a$ fitted on the result. The nominal model $p_{i+1} = p_i + d_i$ is therefore exact on the training data up to clipping into $\mathcal{D}$, so the rollout (6) along which (7) is imposed is the same map that generated the labels. Joint velocities are recorded and used directly as policy inputs. We train the policy of [1] from scratch with a U-Net backbone and a ResNet-18 encoder per view, observation horizon $H_o = 2$, $H = 16$ and $N = 100$, using AdamW with an initial learning rate of 10^{-4} , batch size 64, and 1000 epochs on an NVIDIA RTX 5090 GPU. The checkpoint is frozen and shared by all four compared methods.

Deployment and results. The controller operates at $\Delta t = 100$ ms and replans every $H_e = 4$ actions against a 2 cm obstacle whose center is measured beforehand, using a 6 cm obstacle-side constraint radius and projecting the final $K = 25$ reverse steps. As shown in Fig. 5, which reports one recorded run per method, DP follows the nominal chunk into the obstacle, PACS brakes to a stop on the nominal path and resumes only after the obstacle is removed, and Post-CBF deviates initially but still contacts it. In contrast, our method reshapes the action chunks during denoising

and passes the obstacle while it remains in place. This qualitative comparison illustrates the benefit of adjusting chunk geometry during generation rather than relying on path-preserving braking or a final-step correction.

VI. LIMITATIONS AND CONCLUSION

This paper presented a denoising-time safety filter for diffusion policies based on discrete-time CBFs. By projecting intermediate action chunks and feeding the corrections into subsequent denoising steps, the method enables obstacle-aware action generation without retraining. Across Can, Square, and Lift, the method achieves the highest aggregate joint success-and-safety rate of 42.7% among DP, Post-CBF, PACS, PD, RoS, and Ours, with the best joint rates on Can and Square. Projection-depth and Bounds-only ablations support the benefits of denoising-time correction and obstacle constraints beyond action limiting. UR3 demonstrations illustrate continued motion around an obstacle that remains in place.

However, there are several limitations of our method: (1) In dynamic environments, a longer execution horizon H_e and diffusion inference latency can limit timely responses to moving obstacles, whose motion may invalidate a filtered chunk before the next policy update. (2) Current safety constraints cover only the end effector. Extending the policy to represent whole-body motion in a higher-dimensional action space can make optimization more difficult and lead to slower or less stable training convergence. Alternatively, directly imposing safety constraints on the robot’s full-body geometry requires complex, robot-specific constraint formulations that may limit transferability across platforms. We hope to address these limitations in future work.

REFERENCES

- [1] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. C. M. Burchfiel, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Proceedings of Robotics: Science and Systems*, 2023.
- [2] K.-C. Hsu, H. Hu, and J. F. Fisac, “The safety filter: A unified view of safety-critical control in autonomous systems,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 7, no. 1, pp. 47–72, 2024.
- [3] K. P. Wabersich, A. J. Taylor, J. J. Choi, K. Sreenath, C. J. Tomlin, A. D. Ames, and M. N. Zeilinger, “Data-driven safety filters: Hamilton-jacobi reachability, control barrier functions, and predictive methods for uncertain systems,” *IEEE Control Systems Magazine*, vol. 43, no. 5, pp. 137–177, 2023.
- [4] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, “Control barrier functions: Theory and applications,” in *2019 18th European Control Conference (ECC)*, 2019, pp. 3420–3431.
- [5] A. Agrawal and K. Sreenath, “Discrete control barrier functions for safety-critical control of discrete systems with application to bipedal robot navigation,” in *Proceedings of Robotics: Science and Systems*, 2017.
- [6] J. Thumm and M. Althoff, “Provably safe deep reinforcement learning for robotic manipulation in human environments,” in *2022 IEEE International Conference on Robotics and Automation (ICRA)*, 2022, pp. 6344–6350.
- [7] W. Jung, D. Anthony, U. A. Mishra, N. Ranawaka Arachchige, M. Bronars, D. Xu, and S. Kousik, “RAIL: Reachability-aided imitation learning for safe policy execution,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 3582–3589.
- [8] S. Park, F. Zhang, B. Mirzasoleiman, S. Talebi, and N. Sehatbakhsh, “Your model already knows: Attention-guided safety filter for vision-language-action models,” *arXiv preprint arXiv:2606.09749*, 2026.
- [9] R. Römer, J. Balletshofer, J. Thumm, M. Pavone, A. P. Schoellig, and M. Althoff, “From demonstrations to safe deployment: Path-consistent safety filtering for diffusion policies,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [10] W. Xiao, T.-H. Wang, C. Gan, R. Hasani, M. Lechner, and D. Rus, “SafeDiffuser: Safe planning with diffusion probabilistic models,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [11] J. Zhang, L. Zhao, A. Papachristodoulou, and J. Umenberger, “Constrained diffusers for safe planning and control,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025.
- [12] J.-B. Bouvier, K. Ryu, K. Nagpal, Q. Liao, K. Sreenath, and N. Mehr, “DDAT: Diffusion policies enforcing dynamically admissible robot trajectories,” in *Proceedings of Robotics: Science and Systems*, 2025.
- [13] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 6840–6851.
- [14] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine, “Planning with diffusion for flexible behavior synthesis,” in *Proceedings of the 39th International Conference on Machine Learning*, vol. 162. PMLR, 2022, pp. 9902–9915.
- [15] J. Carvalho, A. T. Le, M. Baierl, D. Koert, and J. Peters, “Motion planning diffusion: Learning and planning of robot motions with diffusion models,” in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 1916–1923.
- [16] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Proceedings of Robotics: Science and Systems*, 2023.
- [17] J. Zeng, B. Zhang, and K. Sreenath, “Safety-critical model predictive control with discrete-time control barrier function,” in *2021 American Control Conference (ACC)*, 2021, pp. 3882–3889.
- [18] A. Singletary, W. Guffey, T. G. Molnar, R. Sinnet, and A. D. Ames, “Safety-critical manipulation for collision-free food preparation,” *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 10954–10961, 2022.
- [19] X. Dai, D. Yu, S. Zhang, and Z. Yang, “Safe flow matching: Robot motion planning with control barrier functions,” *arXiv preprint arXiv:2504.08661*, 2025.